

Data-Driven Problem Solving

The Cooper Union for the Advancement of Science and Art
ME 371 - Fall 2025

Masoud Masoumi

Agenda

1. Imbalanced Datasets

- Causes and challenges
- Resampling techniques

2. Cross-Validation

- K-Fold cross-validation
- Stratified cross-validation

3. Hyperparameter Tuning

- Grid search
- Random search

4. Feature Selection

- Feature importance
- Recursive Feature Elimination (RFE)

Imbalanced Datasets

The Challenge of Imbalanced Data

The Problem:

Imbalanced datasets occur when one class significantly outnumbers the other(s). This is extremely common in real-world applications.

Why It Matters:

- Machine learning algorithms tend to predict the **majority class** more often
- Models achieve misleadingly high accuracy by simply predicting the majority class
- Critical events like fraud detection, disease diagnosis, or equipment failure often represent the minority class

Example Scenarios:

- Credit card fraud: ~0.1% of transactions are fraudulent
- Disease diagnosis: Most patients are healthy
- Manufacturing defects: Most products pass quality control
- Equipment failure: Machines operate normally most of the time

Why Standard Accuracy Fails

Example: Credit Card Fraud Detection

Dataset: 10,000 transactions

- Normal: 9,900 (99%)
- Fraudulent: 100 (1%)

Naive Model: Always predict "Normal"

- Accuracy: 99% ✓
- Fraud detected: 0% ✗

The problem: High accuracy masks complete failure to detect fraud!

Better Metrics for Imbalanced Data:

- **Precision:** Of predicted frauds, how many were actually fraud?
- **Recall:** Of actual frauds, how many did we catch?
- **F1-Score:** Harmonic mean of precision and recall
- **Confusion Matrix:** See the full breakdown of predictions

Causes of Imbalanced Data

1. Biased Sampling

- Data collected from narrow geographical region or time period
- Class distribution may be quite different from true population
- Example: Medical data from a single hospital may not represent broader patterns

2. Measurement Errors

- Incorrect class labels applied to many examples
- Damaged or impaired data collection systems
- Human error in labeling or data entry

3. Nature of the Problem Domain

- Natural occurrence of one class dominates others
- Rare events are genuinely rare (not a sampling issue)
- Example: Equipment failures, fraud, rare diseases
- Process generating minority class observations may be expensive (time, cost, resources)

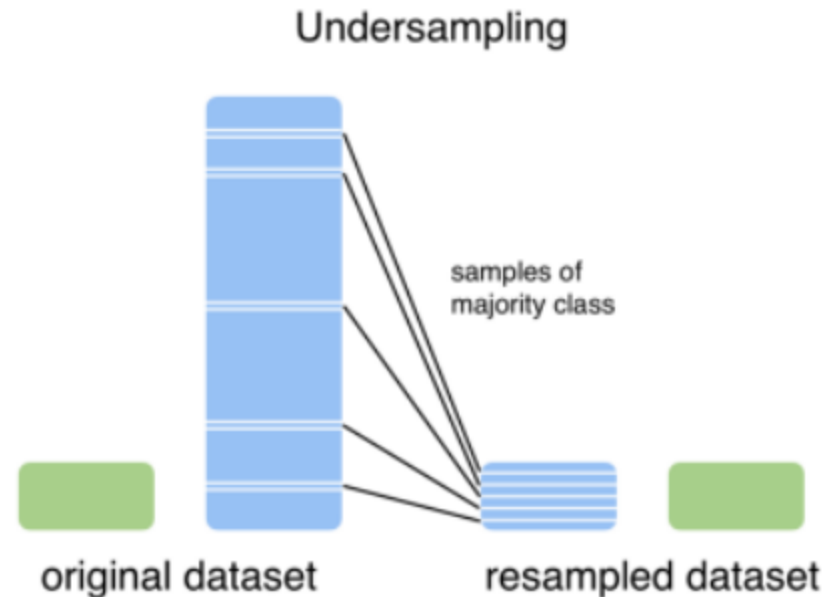
NOTE: Complete Part 1 of the in-class assignment to generate and see the dataset.

Resampling Techniques: Undersampling

Two Main Approaches:

1. Undersampling (Reduce Majority Class)

- Randomly remove samples from majority class
- Balance the dataset by having fewer majority samples
- Risk: May lose valuable information
- When to use: Large datasets where losing data is acceptable



Undersampling Techniques

Random Undersampling

- Randomly remove samples from majority class until balanced
- Simple but may remove important information

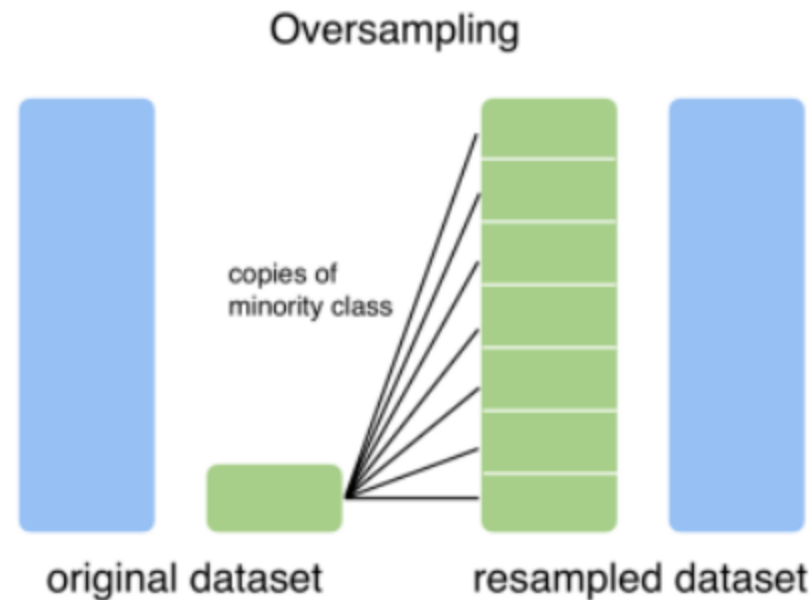
```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(sampling_strategy=0.5) # minority/majority ratio
X_resampled, y_resampled = rus.fit_resample(X_train, y_train)
```


Resampling Techniques: Oversampling

2. Oversampling (Increase Minority Class)

- Add more samples to minority class
- Can duplicate existing samples or generate synthetic ones
- Risk: May lead to overfitting on minority class
- When to use: Small datasets where data is precious



Oversampling Techniques

Random Oversampling

- Duplicate random samples from minority class

```
from imblearn.over_sampling import RandomOverSampler  
  
ros = RandomOverSampler(sampling_strategy=0.5)  
X_resampled, y_resampled = ros.fit_resample(X_train, y_train)
```

Oversampling Techniques

How SMOTE Works:

1. Select a minority sample
2. Find its k nearest minority neighbors
3. Create synthetic sample along the line between the sample and a randomly chosen neighbor

SMOTE (Synthetic Minority Over-sampling Technique)

- Creates synthetic samples by interpolating between existing minority samples
- More sophisticated than simple duplication

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy=0.5)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

NOTE: Complete Part 2 of the in-class assignment to compare resampling techniques.

Cross-Validation

Why Cross-Validation?

The Problem with Single Train/Test Split:

- Performance depends heavily on which samples end up in test set
- One "lucky" or "unlucky" split can mislead us
- Doesn't fully utilize our limited data for training

Cross-Validation Benefits:

- More reliable estimate of model performance
- Uses all data for both training and testing
- Reduces variance in performance estimates
- Helps detect overfitting
- Better use of limited data

Key Principle:

Cross-validation tests how well your model can learn from some data and then predict on data it hasn't seen.

K-Fold Cross-Validation

The Process:

1. **Shuffle** the training data randomly
2. **Split** into K equal-sized groups (folds)
3. **For each unique fold:**
 - Use that fold as test/validation set
 - Use remaining K-1 folds as training set
 - Train model and evaluate performance
 - Record the evaluation score
4. **Summarize** by averaging all K scores



Common Choices:

- K = 5: Good balance of bias and variance
- K = 10: More thorough but more computation
- K = N (Leave-One-Out): Maximum data usage, highest computation

Result: K different performance scores → Average gives robust estimate

K-Fold Cross-Validation Implementation

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LogisticRegression

# Method 1: Quick cross-validation
model = LogisticRegression()
scores = cross_val_score(model, X, y, cv=5) # 5-fold CV

print(f"Scores: {scores}")
print(f"Mean: {scores.mean():.3f}")
print(f"Std: {scores.std():.3f}")

# Method 2: More control
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')

# Method 3: Manual (for learning purposes)
from sklearn.model_selection import KFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []

for train_idx, val_idx in kf.split(X):
    X_train, X_val = X[train_idx], X[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    model.fit(X_train, y_train)
    score = model.score(X_val, y_val)
    scores.append(score)
```

Stratified K-Fold Cross-Validation

The Problem with Standard K-Fold for Imbalanced Data:

Consider a dataset with 95% class 0, 5% class 1:

- Random split might create a fold with NO class 1 samples
- Model trained on that fold never sees minority class
- Evaluation becomes unreliable

Solution: Stratified K-Fold

- Maintains the **same class distribution** in each fold
- Each fold has approximately the same percentage of samples from each class
- Essential for imbalanced datasets

Stratified K-Fold Implementation

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Method 1: Quick stratified CV
model = LogisticRegression()
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skfold, scoring='f1')

print(f"F1 Scores: {scores}")
print(f"Mean F1: {scores.mean():.3f} (+/- {scores.std():.3f})")

# Method 2: Multiple metrics
from sklearn.model_selection import cross_validate

scoring = ['accuracy', 'precision', 'recall', 'f1']
results = cross_validate(model, X, y, cv=skfold, scoring=scoring)

print(f"Accuracy: {results['test_accuracy'].mean():.3f}")
print(f"Precision: {results['test_precision'].mean():.3f}")
print(f"Recall: {results['test_recall'].mean():.3f}")
print(f"F1: {results['test_f1'].mean():.3f}")
```

Hyperparameter Tuning

Parameters vs. Hyperparameters

Model Parameters:

- Learned from the data during training
- Model adjusts these automatically
- Examples:
 - Weights and biases in neural networks
 - Coefficients in linear/logistic regression

Model Hyperparameters:

- Set **before** training begins
- Control the learning process
- Cannot be learned from data
- Must be specified by the user
- Examples:
 - Learning rate, number of epochs, batch size
 - K in K-Nearest Neighbors
 - Network architecture (layers, units)

Common Hyperparameters by Algorithm

K-Nearest Neighbors:

- `n_neighbors` : Number of neighbors (K)
- `metric` : Distance metric (euclidean, manhattan, etc.)
- `weights` : Uniform or distance-based

Neural Networks:

- `learning_rate` : Step size for gradient descent
- `batch_size` : Samples per gradient update
- `epochs` : Number of training iterations
- `hidden_layer_sizes` : Network architecture
- `activation` : Activation function
- `optimizer` : Adam, SGD, RMSprop, etc.

Hyperparameter Tuning Process

Goal: Find the combination of hyperparameters that maximizes model performance

General Process:

1. Define the search space

- List hyperparameters to tune
- Specify range/options for each

2. Choose search strategy

- Grid search (exhaustive)
- Random search (sampling)

3. Select evaluation method

- Use cross-validation (not just train/test split)
- Choose appropriate metric

4. Execute search

- Try different combinations
- Record performance for each

5. Select best configuration

- Retrain on full training set with best hyperparameters

Grid Search

How It Works:

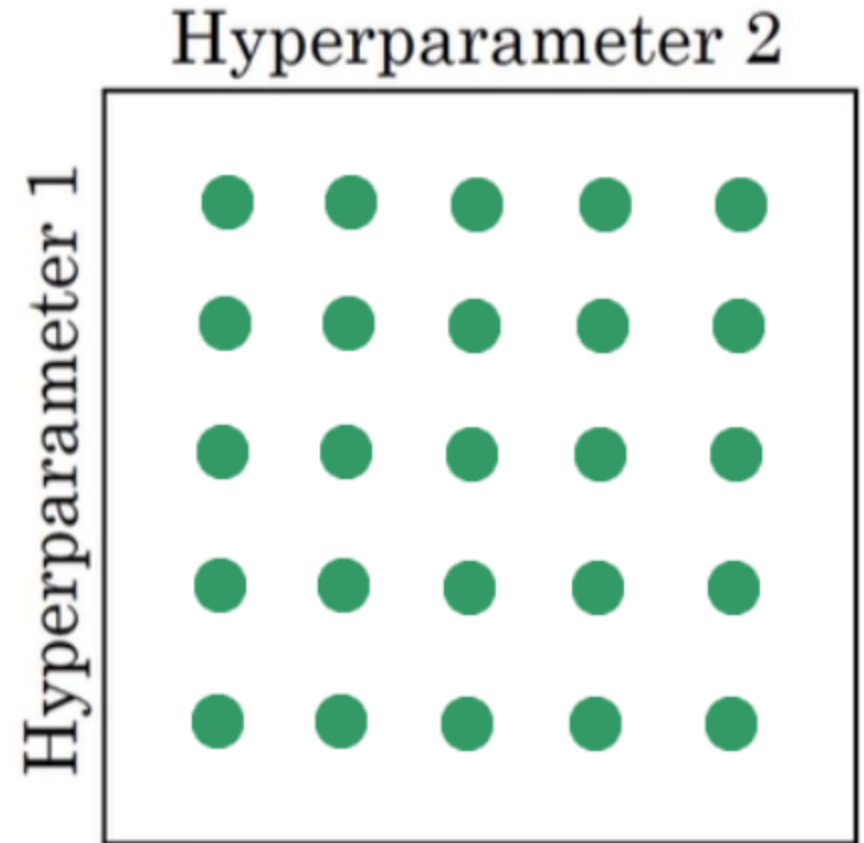
1. Create a **grid** of all possible hyperparameter combinations
2. Train and evaluate model for **every combination**
3. Select combination with best performance

Advantages:

- Exhaustive: tries every combination
- Guaranteed to find best option in grid
- Reproducible results

Disadvantages:

- Computationally expensive
- Exponential growth with more parameters
- May waste time on poor combinations



Grid Search Implementation

```
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier

# Define model
knn = KNeighborsClassifier()

# Define parameter grid
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11],
    'metric': ['euclidean', 'manhattan', 'minkowski'],
    'weights': ['uniform', 'distance']
}

# Setup grid search with cross-validation
grid_search = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    cv=5,                    # 5-fold cross-validation
    scoring='f1',           # Metric to optimize
    n_jobs=-1,              # Use all CPU cores
    verbose=1
)

# Execute search
grid_search.fit(X_train, y_train)

# Results
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best score: {grid_search.best_score_:.3f}")

# Use best model
best_model = grid_search.best_estimator_
```

Random Search

1. How It Works:

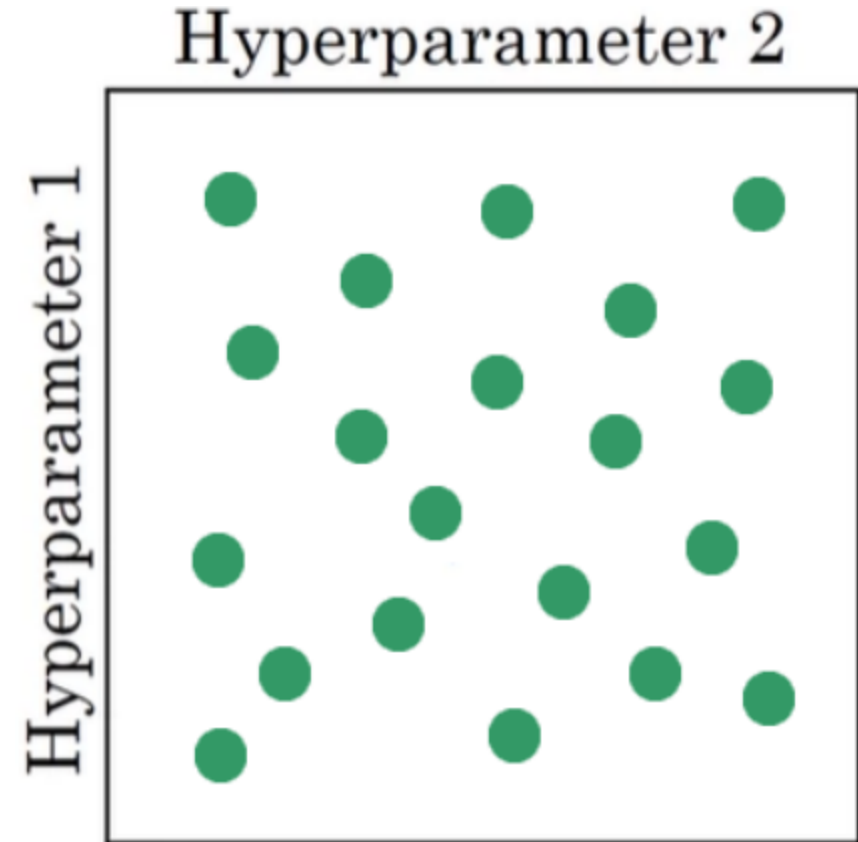
- i. Define **distributions** or ranges for hyperparameters
- ii. **Randomly sample** combinations from these distributions
- iii. Try fixed number of combinations (budget)
- iv. Select combination with best performance

2. Advantages:

- More efficient than grid search
- Can explore wider range of values
- Better for high-dimensional spaces
- Often finds good solutions faster

3. Disadvantages:

- Not exhaustive
- May miss optimal combination
- Results vary between runs (set random_state for reproducibility)



Random Search Implementation

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

# Define model
knn = KNeighborsClassifier()

# Define parameter distributions
param_distributions = {
    'n_neighbors': randint(3, 20),          # Random integers from 3 to 19
    'metric': ['euclidean', 'manhattan', 'minkowski'],
    'weights': ['uniform', 'distance'],
    'p': uniform(1, 2)                      # Random floats from 1 to 3
}

# Setup random search
random_search = RandomizedSearchCV(
    estimator=knn,
    param_distributions=param_distributions,
    n_iter=50,                             # Try 50 random combinations
    cv=5,
    scoring='f1',
    n_jobs=-1,
    random_state=42,
    verbose=1
)

# Execute search
random_search.fit(X_train, y_train)

print(f"Best parameters: {random_search.best_params_}")
print(f"Best score: {random_search.best_score_: .3f}")
```

Grid Search vs. Random Search

Comparison:

Aspect	Grid Search	Random Search
Coverage	Exhaustive within grid	Random sampling
Computation	Can be very expensive	More efficient
Best for	Small parameter space	Large parameter space
Continuous params	Requires discretization	Handles naturally
Reproducibility	Fully reproducible	Needs random_state

Rule of Thumb:

- Grid Search: When you have 2-3 hyperparameters with few options each
- Random Search: When you have 4+ hyperparameters or large value ranges

NOTE: Complete Part 4 comparing Grid Search and Random Search.

Feature Selection

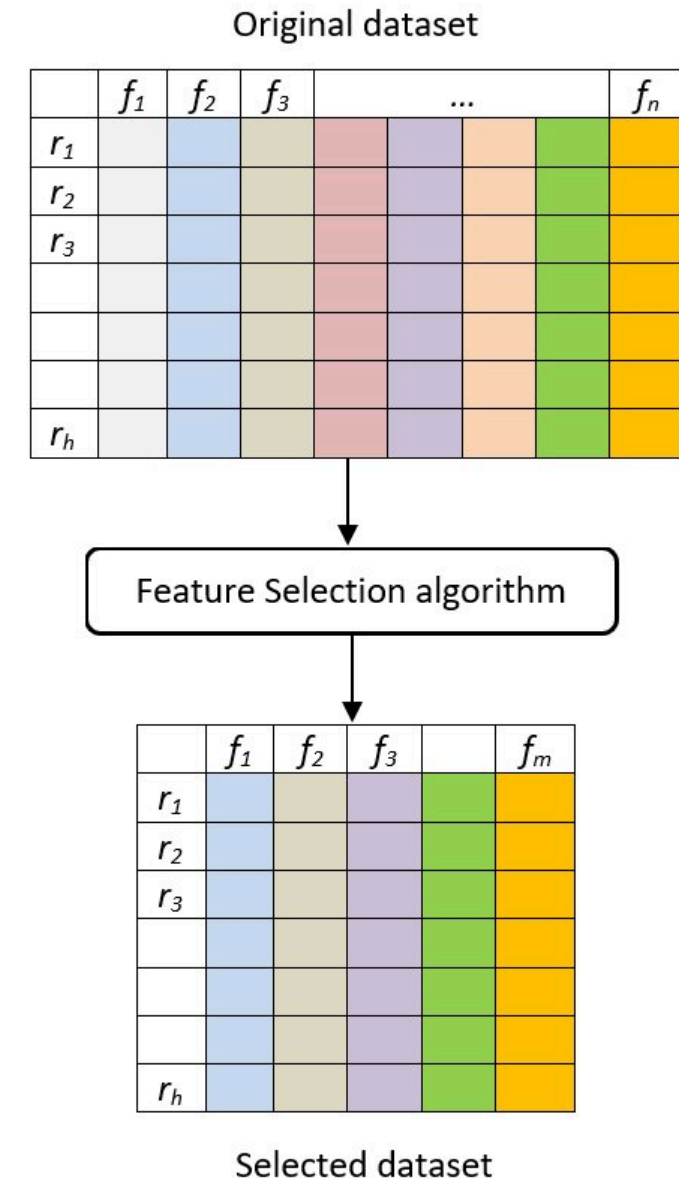
Why Feature Selection Matters

The Challenge: More features $\neq$ Better model

Problems with Too Many Features:

- **Curse of dimensionality:** Model performance degrades in high dimensions
- **Overfitting:** Model learns noise instead of patterns
- **Computational cost:** Training and prediction slow down

Key Principle: The better the features you select, the better the results you achieve.



Recursive Feature Elimination (RFE)

How RFE Works:

1. **Start** with all features
2. **Train** the model
3. **Rank** features by importance
4. **Remove** the least important feature(s)
5. **Repeat** steps 2-4 until desired number of features remains

Key Parameters:

- `n_features_to_select` : How many features to keep
- `step` : How many features to remove per iteration

Advantages:

- Considers feature interactions, Works with any model that has feature importance, Can find optimal subset for specific algorithm

Disadvantages:

- Computationally expensive, Results depend on chosen model

RFE Implementation

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# Define model
model = LogisticRegression()

# Setup RFE
rfe = RFE(
    estimator=model,
    n_features_to_select=5,  # Keep 5 features
    step=1                  # Remove 1 feature per iteration
)

# Fit RFE
rfe.fit(X_train, y_train)

# Get selected features
selected_features = X_train.columns[rfe.support_]
print(f"Selected features: {selected_features.tolist()}")

# Get feature rankings
feature_ranking = pd.DataFrame({
    'feature': X_train.columns,
    'ranking': rfe.ranking_
}).sort_values('ranking')

print(feature_ranking)

# Transforming data to use only selected features
X_train_rfe = rfe.transform(X_train)
X_test_rfe = rfe.transform(X_test)
```

RFECV: RFE with Cross-Validation

Problem with RFE: How do you know how many features to select?

Solution: RFECV

- Performs RFE with cross-validation
- Automatically finds optimal number of features
- Tests different feature subset sizes
- Returns configuration with best CV score

RFECV: RFE with Cross-Validation

```
from sklearn.feature_selection import RFECV

# Setup RFECV
rfecv = RFECV(
    estimator=LogisticRegression(),
    step=1,
    cv=5,                      # 5-fold cross-validation
    scoring='f1',              # Optimization metric
    n_jobs=-1)

# Fit
rfecv.fit(X_train, y_train)

# Results
print(f"Optimal number of features: {rfecv.n_features_}")
print(f"Selected features: {X_train.columns[rfecv.support_].tolist()}")

# Plot number of features vs. CV score
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(rfecv.cv_results_['mean_test_score']) + 1),
         rfecv.cv_results_['mean_test_score'])
plt.xlabel('Number of Features')
plt.ylabel('Cross-validation Score')
plt.title('RFECV: Optimal Number of Features')
```

NOTE: Complete Part 5 of the in-class assignment.

General Workflow

Putting It All Together: Best Practices

Recommended Workflow:

1. Handle Imbalanced Data

- Check class distribution
- Apply appropriate resampling or class weights
- Use stratified splits

2. Feature Selection

- Start with all features
- Use quick filter methods for initial screening
- Apply RFE or RFECV for refinement

3. Hyperparameter Tuning

- Use cross-validation (stratified for imbalanced data)
- Start with random search for exploration
- Refine with grid search around promising regions

4. Final Evaluation

- Train on full training set with best configuration
- Evaluate on held-out test set
- Use appropriate metrics (F1, precision, recall for imbalanced data)

Complete End-to-End Example

```
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import RFECV
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier

# Step 1: initial splitting
print("\nStep 1: Initial train-test split...")
X_pipeline = df.drop('failure', axis=1)
y_pipeline = df['failure']
X_train_p, X_test_p, y_train_p, y_test_p = train_test_split(
    X_pipeline, y_pipeline, test_size=0.2, stratify=y_pipeline, random_state=42
)

# Step 2: scaling
print("Step 2: Scaling features...")
scaler_p = StandardScaler()
X_train_scaled_p = scaler_p.fit_transform(X_train_p)
X_test_scaled_p = scaler_p.transform(X_test_p)

# Step 3: dealing with imbalance with SMOTE
print("Step 3: Applying SMOTE...")
smote_p = SMOTE(sampling_strategy=0.5, random_state=42)
X_train_balanced_p, y_train_balanced_p = smote_p.fit_resample(X_train_scaled_p, y_train_p)

# Step 4: feature selection with RFECV
print("Step 4: Selecting features with RFECV...")
rfecv_p = RFECV(
    estimator=LogisticRegression(max_iter=1000, random_state=42),
    step=1,
    cv=StratifiedKFold(3, shuffle=True, random_state=42), # Use 3-fold for speed
    scoring='f1',
    n_jobs=-1
)
X_train_selected_p = rfecv_p.fit_transform(X_train_balanced_p, y_train_balanced_p)
X_test_selected_p = rfecv_p.transform(X_test_scaled_p)

print(f"    Selected {rfecv_p.n_features_} features")
```

Complete End-to-End Example (cont.)

```
# Step 5: hyper-parameter tuning using Grid Search
print("Step 5: Tuning hyperparameters with Grid Search...")
param_grid_final = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear']
}

grid_final = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid_final,
    cv=StratifiedKFold(5, shuffle=True, random_state=42),
    scoring='f1',
    n_jobs=-1,
    verbose=0
)

grid_final.fit(X_train_selected_p, y_train_balanced_p)

print(f"    Best parameters: {grid_final.best_params}")

# Step 6: final evaluation procedure
print("\nStep 6: Final evaluation on test set...")
final_model = grid_final.best_estimator_
y_pred_final = final_model.predict(X_test_selected_p)

print(f"\nAccuracy: {accuracy_score(y_test_p, y_pred_final):.4f}")
print(f"Precision: {precision_score(y_test_p, y_pred_final):.4f}")
print(f"Recall: {recall_score(y_test_p, y_pred_final):.4f}")
print(f"F1-Score: {f1_score(y_test_p, y_pred_final):.4f}")

# Confusion Matrix
...
```